

ChatGPT - Mid



762df093-024a-4661-9bdc-99167803...

PDF

generate me a detailed understandable explanation on this with proper examples to make me understand better than the pdf and make me exam ready on this topic.

Absolutely! Here's a **detailed, clear, and exam-ready explanation** of Acceptance Criteria, with easy examples and practical tips, going beyond what's in your PDF. This will help you **understand deeply and answer any related question confidently**.

1. Why "Done" is Often Ambiguous

When a team says "we're done" with a user story, what does that really mean?

Example user story:

"As a user, I want to log in with my username and password, so that I can access my account."

The developers build a form, and it lets you log in. Done, right?

But a Product Owner may ask:

- What if someone types the wrong password?
- Is there a "Forgot Password" link?
- Should the system remember users?
- Are there rules for how strong passwords need to be?

Conclusion:

Just making the form isn't enough. There are many hidden requirements.

Acceptance Criteria (AC) solve this by making "done" crystal clear for everyone involved

762df093-024a-4661-9bdc...

2. What Are Acceptance Criteria (AC)?

Definition:

Acceptance Criteria are a list of specific, testable conditions a product or feature must meet to be considered complete.

- They serve as the **Definition of Done** for a user story.
- They act like a contract between the team and the customer or Product Owner.

In Simple Words:

"If all these points are satisfied, we can say this user story is finished. If not, it's not done."

3. Why Are Acceptance Criteria So Important?

- **Clarify Scope:**
AC set boundaries for a story—what's in, what's out—preventing "scope creep" (when features keep getting added).
- **Enable Testing:**
Testers can write cases directly from AC. If it's in the AC, it must be tested.
- **Remove Ambiguity:**
Everyone (devs, testers, designers, product owners) shares the same understanding.
- **Empower Development:**
Developers know exactly what to build and when they're done.

Real-life Example:

If AC says "Password must be at least 8 characters," a 7-character password should always fail—no confusion 762df093-024a-4661-9bdc....

4. Common Formats for Writing Acceptance Criteria

A. Rule-Oriented Checklist

Best for: Simple features.

Looks Like:

- List of rules/conditions that must be true.

Example User Story:

“As a user, I want to search for products, so that I can find what I want to buy.”

Acceptance Criteria Example (Checklist Style):

- Search input is visible on homepage.
- There is a “Search” button.
- User can press Enter or click to search.
- “No results found” appears for empty results.
- Search is not case-sensitive.

If every point passes, the story is done.

B. Scenario-Oriented (Gherkin Format)

Best for: Features involving user interaction or multiple outcomes.

Structure:

- **Given:** Setup or starting state.
- **When:** Action taken by the user.
- **Then:** Expected result.

Example:

User Story: “As a user, I want to log in, so I can access my account.”

Scenario 1: Happy Path

CSS

Given I am on the login page

And I have entered a valid username and password

When I click the "Login" button

Then I am redirected to my account dashboard.

Scenario 2: Unhappy Path

mathematica

Given I am on the login page
And I have entered an invalid username or password
When I click the "Login" button
Then I should see an error message: "Invalid credentials. Please try again."
And I remain on the login page.

Why use this format?

It covers what should happen both when things go right (happy path) and when things go wrong (unhappy path) 762df093-024a-4661-9bdc....

5. How to Brainstorm and Write Acceptance Criteria

A. Start with the Happy Path

- What is the simplest, most common scenario?

Example: User logs in successfully.

B. Explore Unhappy Paths

- What can go wrong?

Example: Wrong password, network error, empty input.

C. Consider Other Factors

- User roles (admin vs. user)
- Performance and security
- Edge cases (e.g., no results)

Tip:

Always write AC as if you're the tester: "If I follow these steps, will I get the expected result?"

6. Characteristics of GOOD Acceptance Criteria

- **Testable & Falsifiable:**

You must be able to check if each criterion is satisfied or not. Avoid vague words.

-  Bad: "Search should be fast."

-  Good: "Search results must load within 2 seconds."
- **Clear & Concise:**
Write in simple, unambiguous language. Everyone should understand.
- **Focus on "What," not "How":**
Don't describe implementation—describe the required outcome.
 -  Bad: "The system will call validate_password()."
 -  Good: "An error is shown if password is less than 8 characters."
- **User Perspective:**
Link criteria to user/business outcomes, not internal tech details 762df093-024a-4661-9bdc....

7. Quick Visual Comparison

Checklist Style	Gherkin Scenario Style
Search bar present	Given: I am on homepage
Search button visible	When: I type a keyword and search
Enter triggers search	Then: Products matching keyword show
"No results" on fail	
Not case-sensitive	

8. Exam-Ready Example & Answer

Question:

Given the user story:

"As a user, I want to reset my password, so that I can access my account if I forget it,"
Write two acceptance criteria in checklist and two in Gherkin format.

Answer:

Checklist Format:

1. A "Forgot Password?" link is visible on the login page.
2. When a valid email is entered, a password reset email is sent.

Gherkin Format:

Scenario 1: Successful Reset

CSS

Given I am on the login page
And I click the "Forgot Password?" link
And I enter a valid registered email
When I submit the form
Then I receive a password reset email.

Scenario 2: Invalid Email

CSS

Given I am on the reset password page
And I enter an email not registered in the system
When I submit the form
Then I see an error message: "Email not found."

9. Summary Table

What are Acceptance Criteria?	How do you write them?	Why are they needed?
List of conditions that define "done"	Checklist or Gherkin	Remove ambiguity, enable testing
Contract for a user story	Focus on user perspective	Define scope, prevent miscommunication

10. Tips for Exams

- Always link your acceptance criteria to user stories and describe outcomes, not technical steps.
 - Give both checklist and scenario (Gherkin) style examples if asked.
 - Emphasize testability, clarity, and relevance to the user.
 - Think of both “happy” and “unhappy” paths.
-

Bottom Line

- **User Story** = *Why* you’re building something.
- **Acceptance Criteria** = *What* needs to be true for the story to be done.

If you can remember the above, and practice writing criteria for real scenarios, **you’re fully exam-ready!**

If you want, I can give you more practice questions or review your own acceptance criteria. Just ask!

now give me questions and answers to practice

Absolutely! Here are **10 practice questions** (with answers) on Acceptance Criteria, mixing both theory and practical application. These are tailored for exam preparation and will help you master the topic.

1. Define “Acceptance Criteria” in your own words.

Answer:

Acceptance Criteria are specific, testable conditions that a software feature must meet to be considered complete. They define what “done” means for a user story and ensure all stakeholders have a shared understanding of the requirements.

2. Why are Acceptance Criteria important in software development? List at least three reasons.

Answer:

1. They clarify the scope and boundaries of a user story.
 2. They provide a basis for testing, allowing QA to create test cases directly from them.
 3. They remove ambiguity by making expectations clear to all team members (developers, testers, product owners).
-

3. Write three acceptance criteria (in checklist format) for the following user story:

"As a user, I want to be able to change my account password, so that I can keep my account secure."

Answer:

- User must enter the current password before changing to a new one.
 - New password must be at least 8 characters long and include one number.
 - A success message is displayed after a successful password change.
-

4. Rewrite the following vague acceptance criterion into a testable one:

"The search results should appear quickly."

Answer:

"Search results must be displayed within 2 seconds after the user submits a query."

5. Describe the difference between checklist-style and scenario (Gherkin) style acceptance criteria.

Answer:

- **Checklist-style** lists specific rules or conditions as bullet points, best for simple features.
- **Scenario-style (Gherkin)** uses a “Given/When/Then” format to describe user interactions and expected outcomes, especially useful for complex workflows or behaviors.

6. Write a Gherkin-style acceptance criterion for this user story:

“As a user, I want to receive an email confirmation after registering, so that I know my registration was successful.”

Answer:

Scenario: Successful Registration

CSS

Given I am on the registration page

And I have entered valid registration details

When I click the “Register” button

Then I should receive an email confirmation at my registered email address.

7. List two characteristics of GOOD acceptance criteria. Give an example of a bad criterion and improve it.

Answer:

Characteristics:

- Testable (clear pass/fail result)
- Clear and concise (easily understood by all)

Bad Example: “The website should be user-friendly.”

Improved: “All forms must have clear labels and error messages must be shown when required fields are empty.”

8. Given this user story, brainstorm both a “happy path” and an “unhappy path” scenario:

“As a user, I want to upload a profile picture, so that my friends can recognize me.”

Answer:

Happy Path:

- User selects a valid image file and uploads it successfully. Profile picture is updated and a success message appears.

Unhappy Path:

- User tries to upload a file that is not an image. An error message appears: “Please select a valid image file.”
-

9. What should acceptance criteria focus on: the ‘how’ or the ‘what’? Explain with an example.

Answer:

Acceptance criteria should focus on the ‘**what**’ (the outcome), not the ‘**how**’ (implementation details).

Example:

- Bad (“How”): The system calls the validateEmail() function.
 - Good (“What”): An error message is displayed if the email address is invalid.
-

10. For the following user story, write one checklist and one Gherkin-style acceptance criterion:

“As an admin, I want to be able to delete user accounts, so that I can manage inactive users.”

Answer:

Checklist:

- Only admins can see the “Delete” button on user profiles.
- A confirmation dialog appears before deleting an account.

Gherkin Scenario:

Scenario: Successful Deletion by Admin

pgsql

Given I am **logged in as** an **admin**

And I am viewing a **user's** profile

When I click the “**Delete**” button

And I confirm the action

Then the **user** account should be removed **from** the **system**

And a success message should be displayed.

If you want more advanced questions, or want to practice by writing your own and getting feedback, let me know!